

Criteria C

Introduction

The product, “Howzat!”, is a cricket coaching tool that aids my client with the following key features.

- The ability for my client to carefully view the original bowling videos of his students on the webpages of the application.
- The ability to submit videos to the application and receive an output ball-tracking video that displays the trajectory of the delivery.
- Pitchmaps that display the length of where the ball landed.

This was possible by building a desktop Flask application that performs Computer Vision processing on the server-side whilst providing an easily useable frontend through HTML webpages. Finally, the ability for my client to access the product through a desktop application meant that the program had no need for a login system and information can be stored locally.

Contents of Documentation

<u>Techniques and concepts used</u>	<u>Description</u>
HTML & CSS & JavaScript	Combination of HTML, CSS and Javascript was used to construct navigable front-end webpages that user utilises to interact with the Flask application
App Routing and JinjaTemplates	App routing and JinjaTemplates was used to link HTML webpages to URLs and add server-side scripting to each URL
HTTP Request Methods	HTTP ‘POST’ and ‘GET’ request methods were used to send and retrieve video data from the user to the server-side scripting of the Flask application
OpenCV2 Video Splitting and Compiling	OpenCV2s video capture function was used to split the video into frames and the video writer function was used to compile frames into an output video.
Yolov5 object detection (AI/ML)	Data Collection: Recording bowling videos to expand training dataset
	Data Labelling: Use of CVAT to annotate data using bounding boxes
	Training: Using annotated dataset to train Yolov5 small model
	Inference: Using Pytorch and Pandas to get object detection coordinates on images
OpenCV2 Drawing	OpenCV2s drawing functions used to draw ball tracking line and bounce spot dots
Text File Storage and Reading	Appending and reading coordinates from text file for master pitchmap
File Handling	Using OS library/CV2 functions to save files and other OS functions to delete
Error Handling	Use of Flash messages in Flask Application for error handling in video upload

Technique 1: HTML + CSS + JavaScript Webpages for Frontend GUI

The application's front end is constructed using four HTML webpages, each of which is linked to a URL and a corresponding Flask function.

As the coach opens the application, he is brought to a landing page where he can choose to submit a video or view the master pitch map.

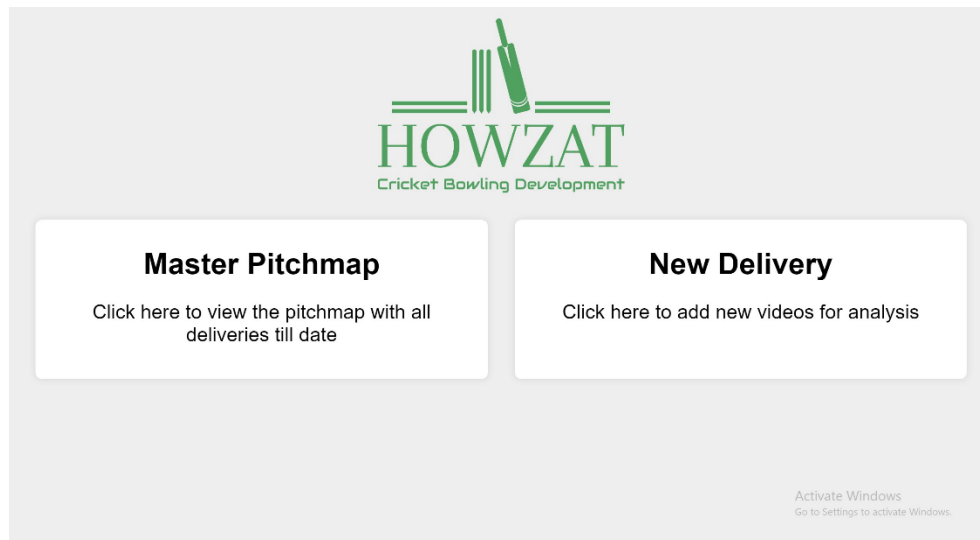


Figure 1 – Landing Page

```
<body>
<!-- Div created and image of logo served from Flask Static Folder -->
<div class="logoDIV">
  
</div>

<!-- Parent Div that will contain two divs for each option within it -->
<div class="parent">

  <!-- OS div: identical divs for both master pitchmap and video submission -->

  <div class="OS" id="older-sessions">
    <h2 class="title">Master Pitchmap</h2>
    <p class="content">Click here to view the pitchmap with all deliveries till date</p>
  </div>

  <div class="OS" id="new-session">
    <h2 class="title">New Delivery</h2>
    <p class="content">Click here to add new videos for analysis</p>
  </div>
</div>

<!-- Scripting used for redirection -->
<script>
  document.getElementById('older-sessions').addEventListener('click', function() {
    window.location.href = '/older_sessions'; //master pitch map flask route in there
  });

  document.getElementById('new-session').addEventListener('click', function() {
    window.location.href = '/new_session'; //video submission flask route in there
  });
</script>
</body>
```

Figure 2 – HTML for Landing page

```
.parent {
  display: flex;
  flex-direction: row;
  align-items: stretch;
  flex-wrap: wrap;
  justify-content: center;
  margin: 20px;
}

.OS {
  background-color: #FFFFFF;
  flex-basis: calc(50% - 40px); /* Stretches the div to 50% of the page leaving 40px for margin */
  flex-direction: row; /* Essential to lining both divs in a row */
  padding: 20px;
  margin: 20px;
  border-radius: 10px;
  box-shadow: 0px 0px 10px rgba(0,0,0,0.1);
  box-sizing: border-box;
  transition: background-color 0.5s, color 0.5s; /* Transition for colour that shows when user hovers */
  cursor: pointer;
  text-align: center;
  min-height: 225px;
}

.OS:hover {
  background-color: #50A060; /* Colour that will show when hovered (colour of logo) */
  color: #fff;
}
```

Figure 3 – Main CSS for Landing page

In Figure 2, the HTML page has one <div> to display the logo and one parent <div> that contains two nested divs: both act as buttons to redirect the coach to other pages (Chris, 2021). The JavaScript 'script' uses the addEventListener method to add a click event to each nested <div>. As the <div> is clicked, the event is triggered and the 'window.location.href' property is used to redirect the user to the corresponding Flask route (Dul, 2020) (Sham et al., 2021).

This method of redirection is used throughout the application. Figure 3 shows the use of CSS to construct and display the divs along with some other stylistic features, such as hover.

If the coach clicks on the “Master Pitchmap” <div>, the coach is redirected to the “/older_sessions” app route that returns the oldersession.html page.

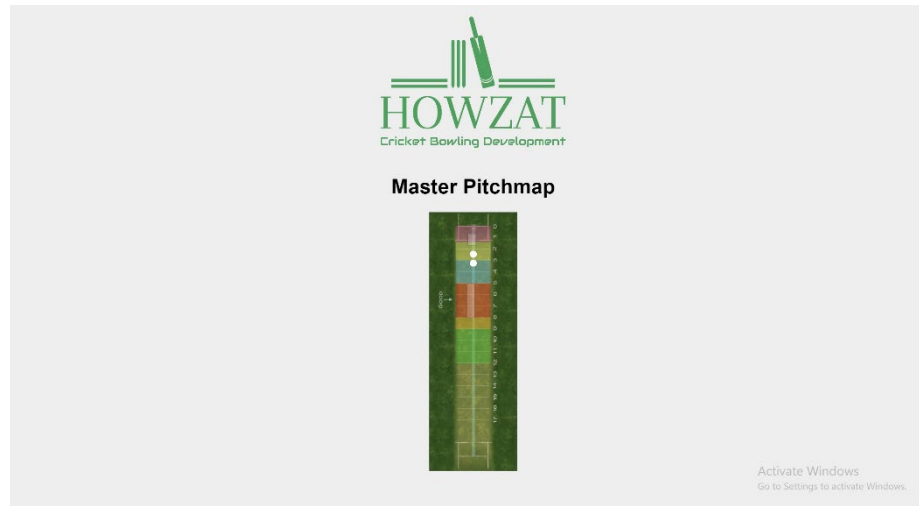


Figure 4 – Oldersession.html Page

```
<body>
  <div class="logoDIV">
    
  </div>

  <div class="image-container">
    <h1 style="text-align: center;">Master Pitchmap</h1>
    
  </div>

  <script>
    document.getElementById('logo').addEventListener('click', function() {
      window.location.href = '/'; //homepage flask route in there
    });
  </script>
```

Figure 5 – HTML for oldsession.html page

The logo is served again from static folder similar to all pages. The “image-container” <div> contains the heading and image of the “master-pitchmap” that is served from the static folder. The same redirection mechanism is used to navigate the user back to the homepage (Dul, 2019). Some CSS was used to ensure the appropriate sizing of the pitchmap on the page (W3Schools, 2020).

If (on the homepage) the coach clicks on the “New Delivery” <div>, he is redirected to the “/new_session” app route that returns the video.html page.

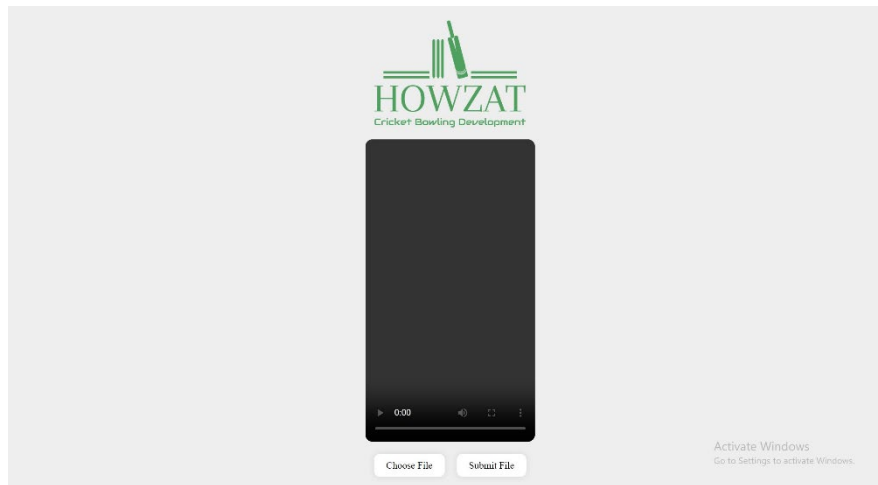


Figure 6 – Videosub.html page

```
<!-- Creating a video player with blank source for now that will be made dynamic -->
<video id="video-player" controls>
  <source src="" type="video/mp4">
</video>

<!-- Creating the buttons for choosing and submitting the file -->
<div id="buttons-container">
  <div class="button" onclick="document.getElementById('fileInput').click();">Choose File</div>
  <div class="button" id="submitButton" type="submit">Submit File</div>
</div>

<!-- Defining input types in the form and the file extensions that will be played and the function that displays video in frame -->
<input type="file" id="fileInput" style="display: none;" accept="video/*,.MOV" onchange="playSelectedFile()">
<script type="text/javascript">
  function playSelectedFile() {
    var videoPlayer = document.getElementById('video-player');
    var fileInput = document.getElementById('fileInput');
    var file = fileInput.files[0];
    var videoUrl = URL.createObjectURL(file);
    videoPlayer.src = videoUrl;
  }
</script>
```

Figure 7 – HTML for Videosub.html page

```
<script>
$(document).ready(function() {
  $('#chooseButton').click(function() {
    $('#fileInput').trigger('click');
  });

  $('#submitButton').click(function() {
    var fileInput = document.getElementById('fileInput');
    var file = fileInput.files[0];
    var formData = new FormData();
    formData.append('file', file);
    $.ajax({
      type: 'POST',
      url: '/new_session',
      data: formData,
      processData: false,
      contentType: false,
    });
  });
});
</script>
```

Figure 8 – HTML scripting for Videosub.html page

The “<video>” tag is used to create a video player where the coach can view the video he will be submitting. Two buttons are created and placed into the button’s container <div> (MozillaDevDocs, 2021) (Figure 7).

When the “Choose File” button is clicked, “document.getElementById(“fileInput”)” is used to select the “<input>” element that allows the coach to select a file from their computer through a pop-up file explorer window as shown below (W3Schools, 2021) (Figure 7):

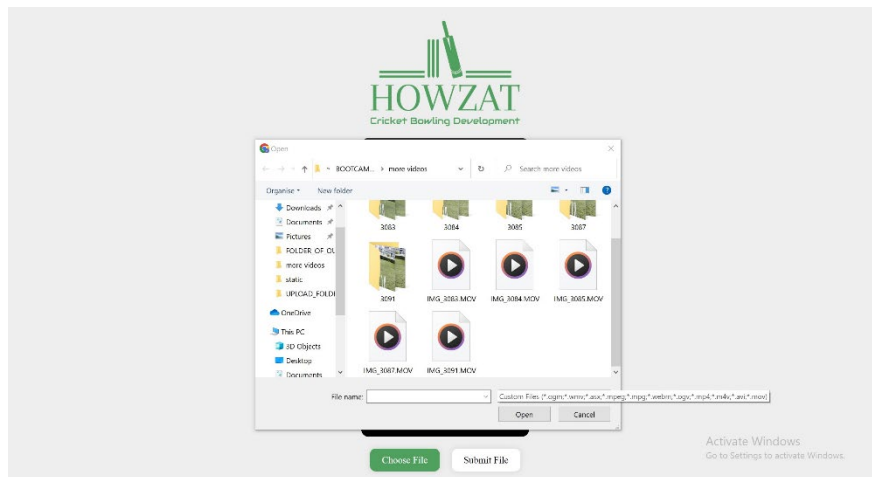


Figure 9 – File explorer window pop up on videosub.html with custom files accepted to allow video file extensions only

Once a file is selected, the “onchange” event is used to call the “playSelectedFile()” function that retrieves the selected video from the input field and displays it in the video frame using an object URL (Patel et al., 2018).

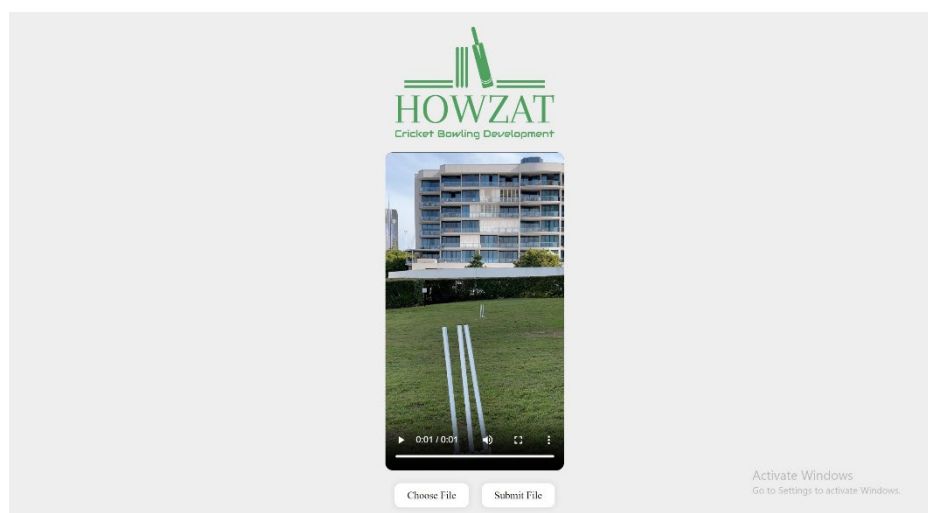


Figure 10 – PlayselectedFile() script function displays the URL object of the coach’s selected video file in the video frame on the videosub.html page

When the “Submit File” button is clicked the “FormData” constructor is used to create an empty object. Then the append() method is used to add the file parameter and the actual video that the user has selected to the object (Figure 8) (StackOverflow, 2019) and prepare it for submission to the app through the form.

Technique 2: HTTP Method Requests for video uploading

From there, an AJAX “POST” request is sent to the “/new_session” app route with the selected video object as the request payload from the HTML page to the server-side processing in Flask (Python Basics, 2021) (Figure 8).

```
#uploading/submitting section
if request.method == 'POST': #Checking if user is trying to submit file using HTTP request
    file = request.files['file'] #retrieves the file object from the from the request that is saved in the 'file' key
    if file: #if the file object exists and is not empty then the following video processing is executed
        destination_path = os.path.join(current_dir, 'UPLOAD_FOLDER', file.filename) #defines filepath of where file will be stored
        file.save(destination_path) #saves file to that location
```

Figure 11 – Flask Backend code in “/new_session” app route that checks for the “POST” HTTP request method from videosub.html

When the Flask application receives the “POST” request, the upload functionality is built by saving the file into the “UPLOAD_FOLDER” in the application directory (Fig 11) (Python Basics, 2021).

Technique 3: OpenCV Video splitting into images

Yolov5 object can only be performed on images. Hence the video submitted by the coach has to be split into images frame-by-frame.

```
#frame splitting of the video that the user submits
vidcapture = cv2.VideoCapture(destination_path) #OpenCV function that opens the video for operations
fps = vidcapture.get(cv2.CAP_PROP_FPS) #gets the frames per second of the video
frame_interval = int(fps / 20) #defines that the video is split 30 frames per second
frame_count = 0 #starting the frame count at the start of the video
success = True #define success as true first to run the while loop till the whole video has been split
#iterate through the frames and save as JPEG images
while success:
    success, frame = vidcapture.read()
    if success and frame_count % frame_interval == 0:
        frame_path = os.path.join(current_dir, 'FOLDER_OF_FRAMES_PATH', f'{frame_count}.jpg') #defines path where frames will be saved
        frame = cv2.rotate(frame, cv2.ROTATE_180) #rotation of each frame (opencv rotates the frame upside down when splitting)
        cv2.imwrite(frame_path, frame) #saves the frame that has been split
        frame_count += 1 #increases count by 1 for next frame in next iteration of while loop
vidcapture.release()
```

Figure 12 – Flask Backend code in “/new_session” that uses OpenCV2 to split the input video and OS to save each frame into a folder in the Flask application directory.

Through the use of OpenCV2’s “VideoCapture()” function and a while loop the video is split into 30 frames per second (explained in comments of Figure 12) (Reshma, 2021).

Technique 4: YOLOv5 Object Detection

In order to perform object detection on the images that have been extracted from the coach's input video, a custom YOLOv5 object detection model has to be trained on bowling footage.

4.1 Data Collection

To ensure maximum accuracy of object detection for the coach, I recorded bowling footage of myself at diverse locations.

4.2 Data Labelling

To label/annotate the stumps and ball in each filmed video, the videos had to be split into images. The code used for this was the exact same as presented in Figure 12. Next, I used an open-source CVAT (Computer Vision Annotation Tool) called Roboflow to draw bounding boxes around the striker stumps, non-striker stumps and ball on each image.



Figure 13 – Computer Vision Annotation Tool labelling process of classes to develop custom training data for YOLOv5 Object Detection Model

Once the images have been annotated, the dataset is exported from Roboflow and is ready for training.



Figure 14 – YOLOv5 Training dataset structure exported from Roboflow

4.3 Training

Once a labelled dataset has been prepared, it was necessary to clone the yolov5 repository to train the model. This was done using git clone with the following commands (Ultralytics Github, N.D):

```
Microsoft Windows [Version 10.0.19045.3086]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Aditya Ghai>cd C:/project/Desktop

C:\project\Desktop>mkdir yolov5_repo      Creating folder for repository
C:\project\Desktop>cd yolov5_repo        Navigating into the newly created folder
C:\project\Desktop\yolov5_repo>git clone https://github.com/ultralytics/yolov5      Cloning repository and installing
Cloning into 'yolov5'...
remote: Enumerating objects: 15807, done.
remote: Counting objects: 100% (38/38), done.
remote: Compressing objects: 100% (36/36), done.
Receiving objects: 100% (15807/15807), 14.58 MiB | 6.55 MiB/s, done.9Receiving objects: 100% (15807/15807), 13.09 MiB | 6.52 MiB/s
Resolving deltas: 100% (10821/10821), done.
```

Figure 15 – Git Cloning of Yolov5 Repository

After cloning the repository and preparing the dataset, the '*train.py*' file (which comes as a part of the repository) is executed through the command terminal with additional arguments as shown below:

```
Microsoft Windows [Version 10.0.19045.3086]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Aditya Ghai>cd C:/project/Desktop/yolov5_repo/yolov5
C:\project\Desktop\yolov5_repo\yolov5>python train.py --img 416 --batch x --epochs y --data dataset.yaml --weights yolov5s.pt --name ball_tracking
```

Figure 15 – Training Command with arguments

Argument	Meaning	Significance
--img 416	Specifies the size of the input image during training in pixels.	Impacts the accuracy and the speed of the model during training.
--batch 16	Specifies the number of images that are processed in a single forward/backward pass.	Adjusted to balance the speed and accuracy of the model. Larger batch size means faster training times but can lead to overfitting and generalization
--epochs 50	Specifies the number of times the entire dataset is passed through the neural network during training.	A high number of epochs can improve accuracy. However, a balance must be determined to avoid overfitting.
--data dataset.yaml	Refers the model to the configuration file that contains information about the dataset file location and class labels	N/A
--weights yolo5s.pt S = small (Small model with fast inference)	Specifies which Yolov5 model weights which model is being trained	Four yolov5 models are compound scaled variants of the same architecture, this parameter defines the size of the model being trained (small, medium, large, and extra-large).

Figure 16 – Training Arguments explained

```

train: weights=yolov5s.pt, cfg=, data=nonevids.yaml, hyp=data\hypos\hyp.scratch-low.yaml, epochs=5, batch_size=1, imgsz=416, rect=False, resume=False, nosave=False, noval=False, noautoanchor=False, noplots=False, evolve=None, bucket=, cache=None, image_weights=False, device=, multi_scale=False, single_cls=False, optimizer=SGD, sync_bn=False, workers=8, project=runs\train, name=ball_trackingtest, exist_ok=False, quad=False, cos_lr=False, label_smoothing=0.0, patience=100, freeze=[0], save_period=1, seed=0, local_rank=-1, entity=None, upload_dataset=False, bbox_interval=-1, artifact_alias=latest
github: YOLOv5 is out of date by 7 commits. Use 'git pull' or 'git clone https://github.com/ultralytics/yolov5' to update.
YOLOv5 v7.0-185-g2334aa7 Python-3.9.0 torch-1.13.1+cpu CPU

Hyperparameters: lr=0.01, lr_f=0.01, momentum=0.937, weight_decay=0.0005, warmup_epochs=3.0, warmup_momentum=0.8, warmup_bias_lr=0.1, box=0.05, cls=0.5, cls_pw=1.0, obj=1.0, obj_pw=1.0, iou_t=0.2, anchor_t=4.0, fl_gamma=0.0, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, degrees=0.0, translate=0.1, scale=0.5, shear=0.0, perspective=0.0, flipud=0.0, fliplr=0.5, mosaic=1.0, mixup=0.0, copy_paste=0.0

Comet: run 'pip install comet_ml' to automatically track and visualize YOLOv5 runs in Comet
TensorBoard: Start with 'tensorboard --logdir runs\train', view at http://localhost:6006/
Overriding model.yaml nc=80 with nc=3

   from  n  params module                                arguments
   --  --  --  --  --
0       -1 1    3520 models.common.Conv              [3, 32, 6, 2, 2]
1       -1 1   18560 models.common.Conv              [32, 64, 3, 2]
2       -1 1   18816 models.common.C3                [64, 64, 1]
3       -1 1   73984 models.common.Conv              [64, 128, 3, 2]
4       -1 2  115712 models.common.C3                [128, 128, 2]
5       -1 1  295424 models.common.Conv              [128, 256, 3, 2]
6       -1 3  625152 models.common.C3                [256, 256, 3]
7       -1 1  1180672 models.common.Conv             [256, 512, 3, 2]
8       -1 1  1182720 models.common.C3                [512, 512, 1]
9       -1 1  656896 models.common.SPPF             [512, 512, 5]
10      -1 1  131584 models.common.Conv             [512, 256, 1, 1]
11      -1 1      0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
12      [-1, 6] 1      0 models.common.Concat              [1]
13      -1 1  361984 models.common.C3                [512, 256, 1, False]
14      -1 1  33024 models.common.Conv              [256, 128, 1, 1]
15      -1 1      0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
16      [-1, 4] 1      0 models.common.Concat              [1]
17      -1 1   90880 models.common.C3                [256, 128, 1, False]
18      -1 1  147712 models.common.Conv              [128, 128, 3, 2]
19      [-1, 14] 1      0 models.common.Concat              [1]
20      -1 1  296448 models.common.C3                [256, 256, 1, False]
21      -1 1  590336 models.common.Conv              [256, 256, 3, 2]
22      [-1, 10] 1      0 models.common.Concat              [1]
23      -1 1  1182720 models.common.C3                [512, 512, 1, False]
24      [17, 20, 23] 1  21576 models.yolo.Detect              [3, [[10, 13, 16, 30, 33, 23], [30, 61, 62, 45, 59, 119], [116, 90, 156, 198, 373, 326]], [128, 256, 512]]

Model summary: 214 layers, 702720 parameters, 702720 gradients, 16.0 GFLOPs

Transferred 343/349 items from yolov5s.pt
optimizer: SGD(lr=0.01) with parameter groups 57 weight(decay=0.0), 60 weight(decay=0.0005), 60 bias
train: Scanning C:\morevids\train\labels.cache... 699 images, 0 backgrounds, 0 corrupt: 100%|██████████| 699/699 [00:00<?, ?it/s]
val: Scanning C:\morevids\valid\labels.cache... 9 images, 0 backgrounds, 0 corrupt: 100%|██████████| 9/9 [00:00<?, ?it/s]

val: Scanning C:\morevids\valid\labels.cache... 9 images, 0 backgrounds, 0 corrupt: 100%|██████████| 9/9 [00:00<?, ?it/s]

AutoAnchor: 3.03 anchors/target, 0.783 Best Possible Recall (BPR). Anchors are a poor fit to dataset , attempting to improve...
AutoAnchor: WARNING Extremely small objects found: 746 of 3957 labels are <3 pixels in size
AutoAnchor: Running kmeans for 9 anchors on 3954 points...
AutoAnchor: Evolving anchors with Genetic Algorithm: fitness = 0.8319: 100%|██████████| 1000/1000 [00:01:00:00, 620.85it/s]
AutoAnchor: thr=0.25: 0.9987 best possible recall, 3.32 anchors past thr
AutoAnchor: n=9, img_size=416, metric_all=0.282/0.831-mean/best, past_thr=0.643-mean: 4.2, 7.13, 13.19, 83.105, 124.75, 66.161, 131.126, 127.179, 145.207
AutoAnchor: Done (Optional: update model *.yaml to use these anchors in the future)
Plotting labels to runs\train\ball_trackingtest\labels.jpg...
Image sizes 416 train, 416 val
Using 0 dataloader workers
Logging results to runs\train\ball_trackingtest6
Starting training for 5 epochs...

   Epoch  GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
   --  --  --  --  --  --  --
0/4      0G      0.1002   0.03284   0.02343      10      416: 100%|██████████| 699/699 [06:16<00:00, 1.86it/s]
   Class  Images  Instances  P      R      mAP50  mAP50-95: 100%|██████████| 5/5 [00:02<00:00, 2.30it/s]
   all      9      26      0.793  0.259  0.187  0.0868

   Epoch  GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
   --  --  --  --  --  --  --
1/4      0G      0.08221   0.02904   0.007166      10      416: 100%|██████████| 699/699 [05:53<00:00, 1.97it/s]
   Class  Images  Instances  P      R      mAP50  mAP50-95: 100%|██████████| 5/5 [00:01<00:00, 2.59it/s]
   all      9      26      0.739  0.479  0.476  0.178

   Epoch  GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
   --  --  --  --  --  --  --
2/4      0G      0.0722   0.02734   0.005222      9      416: 100%|██████████| 699/699 [05:53<00:00, 1.98it/s]
   Class  Images  Instances  P      R      mAP50  mAP50-95: 100%|██████████| 5/5 [00:01<00:00, 2.59it/s]
   all      9      26      0.73  0.516  0.426  0.15

   Epoch  GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
   --  --  --  --  --  --  --
3/4      0G      0.06284   0.02584   0.004906      5      416: 100%|██████████| 699/699 [05:59<00:00, 1.95it/s]
   Class  Images  Instances  P      R      mAP50  mAP50-95: 100%|██████████| 5/5 [00:01<00:00, 2.67it/s]
   all      9      26      0.98  0.649  0.663  0.34

   Epoch  GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size
   --  --  --  --  --  --  --
4/4      0G      0.05627   0.02512   0.004018      4      416: 100%|██████████| 699/699 [05:51<00:00, 1.99it/s]
   Class  Images  Instances  P      R      mAP50  mAP50-95: 100%|██████████| 5/5 [00:01<00:00, 2.80it/s]
   all      9      26      0.795  0.593  0.613  0.363

5 epochs completed in 0.502 hours.
Optimizer stripped from runs\train\ball_trackingtest6\weights\last.pt, 14.3MB
Optimizer stripped from runs\train\ball_trackingtest6\weights\best.pt, 14.3MB

Validating runs\train\ball_trackingtest6\weights\best.pt...
Fusing layers...
Model summary: 157 layers, 7018216 parameters, 0 gradients, 15.8 GFLOPs
   Class  Images  Instances  P      R      mAP50  mAP50-95: 100%|██████████| 5/5 [00:01<00:00, 2.86it/s]
   all      9      26      0.796  0.593  0.613  0.363
   Ball      9      8      1      0      0      0
   Stumps      9      9      0.636  1      0.935  0.513
   stumps      9      9      0.753  0.778  0.904  0.575

Results saved to runs\train\ball_trackingtest6

```

Figure 17 – Example Bogus run of training

After all epochs of training have been completed, YOLOv5 saves the best-performing configuration of weights (of the neural network) into a file called ‘Best.pt’. (Ultralytics Github, N.D)

4.4 Inference

The “Best.pt” file is used to draw inferences on the images that were extracted from the video which the coach submits.

```
#Once the video is split into frames - object detection can be performed on each frame
#Using Pytorch to load the best weights model for object detection
model = torch.hub.load('ultralytics/yolov5', 'custom', 'C://project//Desktop//yolov5//runs//train//homeD//weights//best.pt')
#fixing of frame number intake by sorting the list
frameslist = os.listdir(os.path.join(current_dir, 'FOLDER_OF_FRAMES_PATH')) #saves a list of all the file names of jpegs
framesnumber = len(frameslist) #gets the length of the list - this will be the number of images that are in the folder
sortedframeslist = [] #creates the new list where the images will be sorted
for u in range(0,framesnumber): #creating for loop from 0 to the number of frames
    sortedframeslist.append(u) #append each number to the sortedlist
#add .jpg to each of the numbers to form the sorted list of file names
sortedframeslist = [str(fname) + '.jpg' for fname in sortedframeslist]
```

Figure 18 – Flask backend code in “/new_session” app route that loads the YOLOv5 object detection model and creates a list of sorted file names

Once a list containing sorted file names was ready, a for loop was used to iterate through all the frames in the input video sequentially, to perform object detection on each image.

```
#machine learning inference of finding stumps and ball
for image in sortedframeslist: #for loop through all the file names in the sorted filename list
    img = os.path.join(current_dir, "FOLDER_OF_FRAMES_PATH", image) #defines file path of image that from for loop
    results = model(img) #Pytorch function that performs the object detection using the loaded weights
    image_objs = len(results.pandas().xyxy[0]) #using length to define how many objects were detected in image
    for i in range(0,image_objs): #for loop from 0 to the number of objects used to extract coordinates of specific object
        if (results.pandas().xyxy[0]['name'][i]) == 'Ball':
            xmin = (results.pandas().xyxy[0]['xmin'][i])
            ymin = (results.pandas().xyxy[0]['ymin'][i])
            xmax = (results.pandas().xyxy[0]['xmax'][i])
            ymax = (results.pandas().xyxy[0]['ymax'][i])
            centerx = ((xmin+xmax)/2) #center x coordinate of ball
            centery = ((ymin+ymax)/2) #center y coordinate of ball
            ballx.append(centerx) #appended into separate parallel arrays
            bally.append(centery)
            #####
        for q in range(0,image_objs):
            if (results.pandas().xyxy[0]['name'][q]) == 'Stumps':
                yminS = (results.pandas().xyxy[0]['ymin'][q])
                ymaxS = (results.pandas().xyxy[0]['ymax'][q])
                heightS = (ymaxS - yminS) #subtracting max and min to get pixel height
                stumps_heights.append(heightS) #storing height of stumps
                baseS.append(ymaxS) #storing base coordinate
                #####
            for z in range(0,image_objs):
                if (results.pandas().xyxy[0]['name'][z]) == 'stumpNS':
                    yminNS = (results.pandas().xyxy[0]['ymin'][z])
                    ymaxNS = (results.pandas().xyxy[0]['ymax'][z])
                    heightNS = (ymaxNS - yminNS) #subtracting max and min to get pixel height
                    stumpsNS_heights.append(heightNS) #storing height of stumps
                    baseNS.append(ymaxNS) #storing base coordinate
```

Figure 19 – For Loop iterating through all split images and performing object detection on each frame with comment explanations

The model(img) Pytorch function is used to draw an inference on the image and Pandas is used to retrieve the following Pandas dataframe (Pytorch, 2016):

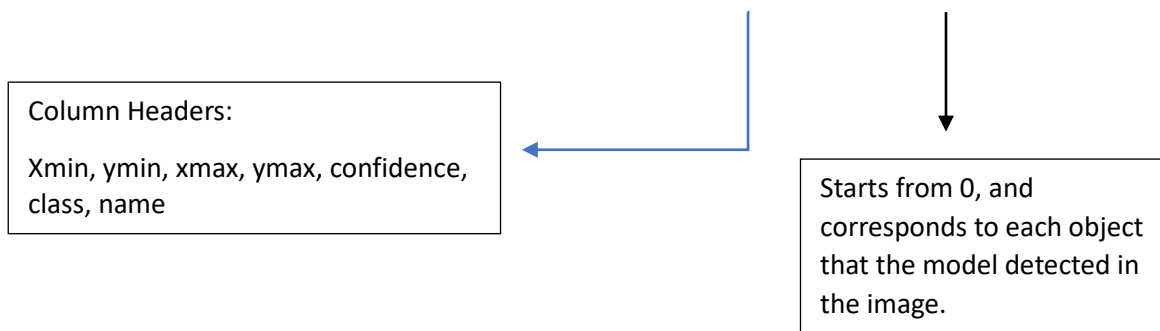
```
results.pandas().xyxy[0]
```

#	xmin	ymin	xmax	ymax	confidence	class	name
# 0	749.50	43.50	1148.0	704.5	0.874023	0	person
# 1	433.50	433.50	517.5	714.5	0.687988	27	tie
# 2	114.75	195.75	1095.0	708.0	0.624512	0	person
# 3	986.00	304.00	1028.0	420.0	0.286865	27	tie

Figure 20 – Example pandas data structure of pixel coordinates of bounding box produced when object detection done on an image

To traverse the dataframe and retrieve the values, the following Pandas syntax is used:

```
results.pandas().xyxy[0]['column header'][row number]
```



Once the desired coordinates have been retrieved, they are stored in global lists that can be accessed for analysis.

For the ball coordinates, the center-x and center-y coordinates of the bounding boxes are stored in two parallel arrays. Whereas for the stumps, their heights and base coordinates were stored in separate arrays (Figure 19). These will be necessary to develop the ball tracking videos and Pitchmaps.

Technique 5: OpenCV2 Drawing

5.1 Ball Tracking

To develop the ball-tracking video two OpenCV2 drawing functions will be performed on each frame:

1. `cv2.polylines()`: draws a white line through all the coordinates of the ball in the “a” array (zipped array of x and y coordinates of ball) on every single frame (OpenCV2Docs, 2014).
2. `cv2.circle()`: draws a red circle at the co-ordinates where the ball has bounced (OpenCV2Docs, 2014).

The bounce coordinate can be found using the “max()” function on the y-coordinates array, as the highest y-coordinate will be where the ball is lowest in the frame, which will be when it makes contact with the ground.

```
#creating 2 dimensional array with co-ordinates of ball in each image
a = (list(zip(ballx, bally)))
a = np.array(a) #making it an numpy array
a = a.reshape((-1,1,2)) #reshaping it
colour = (255, 255, 255) #choosing colour of line of ball path - white
thickness = 3 #thickness of line
isClosed = False #closing line set to false to have a single line and not a loop
#highest y-cord will be where ball bounces
#hence this gives index of the element which has the co-ordinates to where the ball bounces
indexofbounce = bally.index(max(bally))
bouncex = ballx[indexofbounce] #storing x coordinate of where the ball bounces
boucey = bally[indexofbounce] #storing y coordinate of where the ball bounces
#Video Drawing logic
for j in sortedframeslist: #For loop through the sorted list of filenames
    impath = os.path.join(current_dir, "FOLDER_OF_FRAMES_PATH", j) #path of the image frame
    final_img = cv2.imread(impath) #Using opencv2 to open the image to edit it
    cv2.polylines(final_img, np.int32([a]), isClosed, colour, thickness) #Draw line through all coordinates of ball
    cv2.circle(final_img, (int(bouncex), int(boucey)), radius=15, color=(0,0,255), thickness=-1) #Draw circle where ball bounces
    cv2.imwrite(os.path.join(current_dir, 'FOLDER_OF_OUTPUT_FRAMES_PATH', j), final_img)
    #final image saved in folder of output frames path
    #the for loop allows me to draw the line and dot on each image so it is present throughout the complete video
```

Figure 21 – Drawing ball tracking and bounce line for the ball tracking video

As all the images are now drawn on, the following OpenCV2 code was used to compile all the frames back into an output video for the user.

```
#compiling frames of ball tracking all together for an output video
final_images_folder = os.path.join(current_dir, "FOLDER_OF_OUTPUT_FRAMES_PATH") #defining path where images are stored
fm = cv2.imread(os.path.join(final_images_folder, sortedframeslist[0])) #opening the first image in the folder
height, width, _ = fm.shape #shaping the image
output_path = os.path.join(current_dir, "static", "output.mp4") #defining where the output video will be saved
video = cv2.VideoWriter(output_path, cv2.VideoWriter_fourcc(*'MPV4'), 30, (width, height))
#defining the videowriter opencv function to compile all the images together
for l in sortedframeslist:
    img_path = os.path.join(final_images_folder, l)
    fm = cv2.imread(img_path) #reading the files
    video.write(fm) #writing them in the defined video writer function above
cv2.destroyAllWindows() #destroying windows of opencv2 operation
video.release() #video being released and saved at this point
```

Figure 22 – Drawing ball tracking and bounce line for the ball tracking video

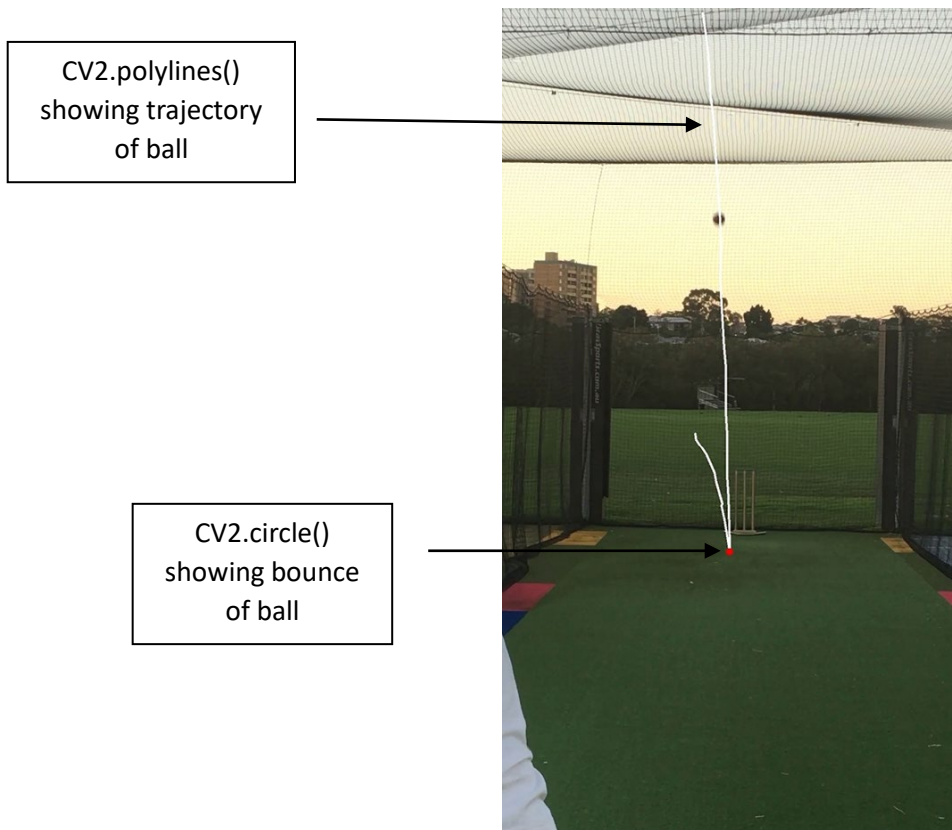


Figure 23 – Jpeg shows the result of CV2.circle() and CV2.polylines()

5.2 PitchMaps

For developing the pitchmaps, mathematic logic was coded into Python to calculate the meter distance between the ball and the striker stumps.

```
#pitchmap logic
stumpH = sum(stumps_heights)/(len(stumps_heights)) #average height of the striker stumps
NSstumpH = sum(stumpsNS_heights)/(len(stumpsNS_heights)) #average height of non-striker stumps
NS_Base = sum(baseNS)/(len(baseNS)) #average length of the base of non-striker stumps
S_Base = sum(baseS)/(len(baseS)) #average length of the base of the striker stumps
stump_to_stump_dist = (NS_Base - S_Base) #pixel distance between the base of the striker and non striker stumps
###cords are (0, NS stumps height) and (base-to-base-distance, farstumps height)
m = (NSstumpH - stumpH)/(-(stump_to_stump_dist)) #defines the gradient of the function of stump height vs distance
#substitute the pixel distance the ball is away from the non-striker stumps into the function
k = m*((NS_Base-bouncey)-stump_to_stump_dist) + stumpH #height of stump if reconstructed at ball
meter_dist = (((k)/NSstumpH))*20.12 #calculation of meter distance using propotion of stump
```

Figure 24 – Mathematic Logic to calculate distance between far stumps and ball

This logic of calculating distances in a perspective-distorted image with two reference objects of equal size (stumps) by the reconstruction of the reference object at the ball was adapted from an in-depth article written by S. Khandelwal in 2021 regarding the “Distance estimation of various objects present in a depth image using object detection”.

After calculating the meter distance `cv2.circle()` is used again to draw a dot on the pitchmaps to indicate the length of the ball on both current delivery and master pitchmap.

```
#now that we have how far the ball bounces away from the striker we have to draw it on the pitchmap
Y_pixel_cal = 50*(meter_dist) + 60 #approximation based on the dimensions of the pitchmap
pitchmap_current = os.path.join(current_dir, "static", "pitchmap.png") #defines path of original pitchmap
pitch_img = cv2.imread(pitchmap_current) #opening image for opencv2 to edit the pitchmap
cv2.circle(pitch_img, (int(191), int(Y_pixel_cal)), radius = 15, color=(255,255,255), thickness=-1)
#White circle drawn on pitchmap at where the ball lands
cv2.imwrite(os.path.join(current_dir, "static", "balln.png"), pitch_img) #saves the pitchmap in the static folder for display
```

Figure 25 – OpenCV2 drawing logic for Pitchmap of previous delivery

Technique 6: Text File reading and writing

For the master pitchmap, there needed to be some way to store the coordinates of deliveries. The use of a database would be excessive for such little information. Hence a locally stored text file was used.

```
#Saving co-ordinates of where the ball should be on the pitchmap to a coordinates txt file to construct the master pitchmap
with open('coordinates.txt', 'a') as f: #opening the coordinates file in append mode to add coordinates to it
    f.write('191,' + str(Y_pixel_cal) + '\n') #adding the coordinates of the delivery in the submitted video

with open('coordinates.txt', 'r') as m: #opening the coordinates file in reading mode
    lines = m.readlines() #using read lines to store all lines in a file
    for line in lines: #using for loop to iterate through all the lines
        x, y = line.strip().split(',') #splitting each line into x and y
        cordsy.append(int(float(y))) #storing y coordinate into another list
masterpitchmap_path = os.path.join(current_dir, "static", "master-pitchmap.png") #defining the master pitchmap path
master_pitchmap_img = cv2.imread(masterpitchmap_path) #opening the master pitchmap for performing operations
for cord in cordsy: #for loop through all the y-coordinates
    cv2.circle(master_pitchmap_img, (int(191), int(cord)), radius=15, color=(255,255,255), thickness=-1) #add circle at each point
cv2.imwrite(masterpitchmap_path, master_pitchmap_img) #saving the master pitchmap by overwriting the old file
```

Figure 26 – Appending and Reading coordinates from text file

First, the text file is opened, and the coordinates for drawing are appended to it. Next, the text file is opened, and the coordinates are retrieved and stored in a list (NTU, 2017). Then a for loop is used to iterate all coordinates and draw them on the pitchmap using `cv2.cirlce()`.

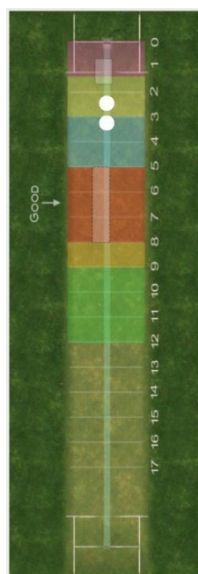


Figure 27 – Master pitchmap
result of two deliveries

Technique 7: File Handling

File handling was primarily performed through Python's OS library and consisted of saving and deleting video and JPEG files.

```
#deleting any previous video that is stored in the upload folder
uploadpath = os.path.join(current_dir, 'UPLOAD_FOLDER') #getting path of upload folder
uploadedfiles = os.listdir(uploadpath) #using os.listdir to produce list of files within the upload folder
for pastvideo in uploadedfiles: #for loop that goes through each file
    pastvideo_path = os.path.join(uploadpath, pastvideo) #exact file path of any video in the folder
    if os.path.isfile(pastvideo_path): #checking if file exists
        os.remove(pastvideo_path) #deleting the file permanently
```

Figure 28 – Using OS functions to delete video files stored within the upload folder (Rigoulet, 2023)

```
#deleting Image files from previous video submission
frame_folder = os.path.join(current_dir, 'FOLDER_OF_FRAMES_PATH') #defines frame folder path
output_frame_folder = os.path.join(current_dir, 'FOLDER_OF_OUTPUT_FRAMES_PATH') #defines output frame folder path
try:
    for file_image_A in os.listdir(frame_folder): #for loops through list of all file names within folder
        fpA = os.path.join(frame_folder, file_image_A) #joins current image in iteration and folder pathway
        if os.path.isfile(fpA): #os.path.isfile() does another check to make sure that a file with that desintation path
            os.remove(fpA) #deleting the file permanently

    #exactly same processes repeated for images that have been drawn on just different folder
    for file_image_B in os.listdir(output_frame_folder):
        fpB = os.path.join(output_frame_folder, file_image_B)
        if os.path.isfile(fpB):
            os.remove(fpB)
except Exception as e:
    print(f"Error occurred while deleting files: {e}")
```

Figure 29 – Using OS functions to delete images that are split from videos (Rigoulet, 2023)

```
cv2.imwrite(masterpitchmap_path, master_pitchmap_img) #saving the master pitchmap by overwriting the old file
```

```
cv2.imwrite(os.path.join(current_dir, "static", "balln.png"), pitch_img) #saves the current ball pitchmap in the static folder for display
```

```
cv2.imwrite(frame_path, frame) #saves the frame that has been split
```

```
cv2.imwrite(os.path.join(current_dir, 'FOLDER_OF_OUTPUT_FRAMES_PATH', j), final_img)
#final image saved in folder of output frames path
```

Figure 30 – Using Cv2.imwrite() functions to save images that are split from videos

Technique 8: Error Handling in Video Submission

The video submission required two phases of error checking: when the video is too long, or when the user has not selected a file.

```
frameslist = os.listdir(os.path.join(current_dir, 'FOLDER_OF_FRAMES_PATH')) #defines list that stores file names of images
framesnumber = len(frameslist) #takes length of list to determine how many frames have been generated from video
#since the video is split 30fps, 150 frames is equivalent of 5 seconds
if framesnumber >= 150: #condition of 5 seconds
    flash('Submission unsuccessful. Please submit a video shorter than 5 seconds.', 'error') #flashing error message on redirected page
    return jsonify({'redirect': url_for('new_session')}) #restarting the url allowing the user to submit a new video
```

Figure 31 – Error Handling for Video length that is greater than 5 seconds (Javapoint, 2020)

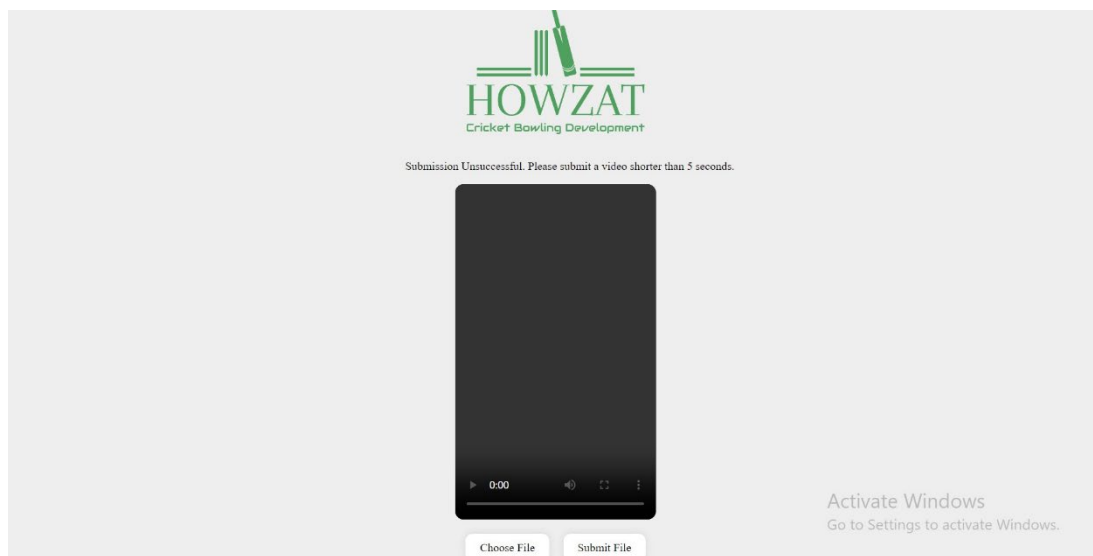


Figure 32 – HTML Flash message displayed on submission of a file greater than 5 seconds.

Additionally, the flask request methods have been set up so that the browser displays an error message if the user clicks the submit button without selecting a file.

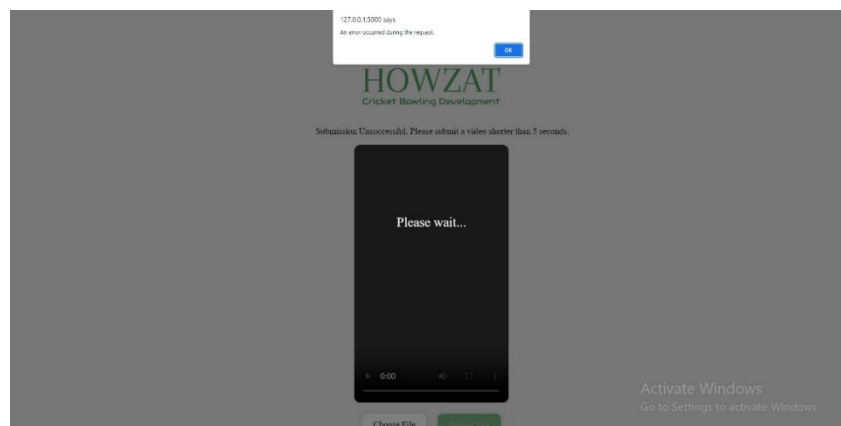


Figure 32 – HTML page has the browser displaying an error message if the user hasn't selected a file

Word Count: 1012 words

References/Bibliography

Appending and Reading from text file – storage of coordinates for master pitch map

Retrieved from

https://www3.ntu.edu.sg/home/ehchua/programming/webprogramming/Python_FileText.html

OpenCV documentation for drawing lines, circles and text onto the image

Retrieved from https://docs.opencv.org/4.x/d6/d6e/group__imgproc__draw.html

Video player HTML & CSS development

Retrieved from <https://developer.mozilla.org/en-US/docs/Web/HTML/Element/video>

Styling and creation of divs

Chris, K. (2021). HTML div – what is a div tag and how to style it with CSS. Retrieved from <https://www.freecodecamp.org/news/html-div-what-is-a-div-tag-and-how-to-style-it-with-css/>

Flask Application serving of static files to a webpage

Dul. (2019). Retrieved from <https://kanchanardj.medium.com/how-to-add-images-to-html-in-a-flask-app-4dbcc92e3aeb>

Javascript redirection through the use of a button

Dul. (2020). Retrieved from <https://kanchanardj.medium.com/redirecting-to-another-page-with-button-click-in-python-flask-c112a2a2304c>

Python code for flashing error message in video submission

Javapoint. Retrieved from <https://www.javatpoint.com/flask-flashing>

Turning video into Object URL for displaying in video frame

Mike. (2019). Retrieved from <https://stackoverflow.com/questions/53950848/change-video-source-to-blob-objecturl>

Developing the playselectedfile() function to display selected video in webpage

Patel, R., Kaiido, & Fabian. (2018). Retrieved from <https://stackoverflow.com/questions/36035721/how-can-i-set-preview-of-video-file-selecting-from-input-type-file>

Researching HTTP methods for video file upload mechanism

Python Basics. Retrieved from <https://pythonbasics.org/flask-http-methods/>

Splitting input video into frames

Reshma. (2021). Retrieved from <https://www.scaler.com/topics/cv2-videocapture/>

Using OS library for File Handling

Rigoulet, X. (2023). Retrieved from <https://learnpython.com/blog/delete-file-in-python/>

Onclick event listeners for redirection

Sham, W. (2021). Retrieved from <https://stackoverflow.com/questions/6348494/addeventlistener-vs-onclick>

Video upload of object through HTML form

W3Schools. Retrieved from https://www.w3schools.com/jsref/dom_obj_fileupload.asp

Positioning and sizing of Images in HTML

W3Schools. Retrieved from https://www.w3schools.com/html/html_images.asp

Pitch map bounce calculation using object detection

Khandelwal, S. (2021). Distance estimation of various objects present in an image from camera when they were captured... Retrieved from <https://khandewalshivam.medium.com/distance-estimation-of-various-objects-present-in-an-image-from-camera-when-they-were-captured-64453847a3a1>

Yolov5 inference results Pandas Data structure

(N.d.). Retrieved from https://pytorch.org/hub/ultralytics_yolov5/

Yolov5 Training on custom dataset documentation

Ultralytics. (n.d.). Ultralytics Yolov5 Documentation. Retrieved from <https://github.com/ultralytics/yolov5>